Reinforcement Learning for Restaurant Inventory Management

Arnav Agarwal (aa5507), Maxime Mobayed (mlm2395), Xiyuan Chen (xc2673)
Columbia University

December 2025

Abstract

We present a deep reinforcement learning framework for restaurant inventory management of perishable goods. A PPO agent with a permutation-invariant DeepSets architecture jointly optimizes daily ordering for multiple ingredients, balancing profit, spoilage, and stockouts under budget and capacity constraints. The policy uses LogNormal action distributions anchored to LSTM demand forecasts, ensuring positive orders while capturing forecast uncertainty. Trained across 40 restaurants and evaluated on 10 held-out restaurants, the learned policy achieves the highest profit ($22K vs $18K for the best heuristic) while reducing spoilage by 77% compared to classical Order-Up-To policies (50 vs. 215 units). These results demonstrate that deep RL can effectively learn long-horizon trade-offs in high-dimensional perishable inventory domains where analytical methods are intractable.

1 Introduction and Motivation

Restaurants face complex inventory challenges: perishable ingredients with heterogeneous shelf-lives, stochastic dish-level demand, and tight budget/storage constraints. Poor coordination leads to stockouts, waste, and budget shortfalls.

This project develops an RL framework for restaurant-scale perishable inventory management. The environment simulates daily review with age-bucket tracking, first-in-first-out (FIFO) consumption, Poisson demand, and budget/capacity constraints [1]. The policy architecture uses DeepSets [2] for permutation-invariant encoding and shared-parameter multi-agent formulation where each ingredient is an agent sharing network weights [1]. A LogNormal output distribution ensures positive order quantities. Demand forecasts from an LSTM ensemble [3] are integrated into observations.

2 Background and Related Work

Classical inventory theory has extensively studied stochastic demand for perishable goods management, using analytical methods such as base-stock policies, newsvendor formulations, and finite-horizon dynamic programming [3]. These methods provide strong results in low-dimensional settings, but depend on simplified dynamics and do not scale well to systems with many interacting perishable items, lead times, and heterogeneous shelf-lives [4]. Recent findings in deep reinforcement learning (DRL) have encouraged learning-based methods for inventory optimization, with actor-critic methods able to handle nonlinear demand processes as well as long-horizon decision-making coupling [3][5]. Proximal Policy Optimization (PPO) has particularly shown to be a stable environment for high-dimensional continuous control and is widely used in operations-focused RL

applications. Currently, most DRL studies focus on environments with a fixed item set and are not designed to generalise across different operational environments, like the multi-restaurant agent we intended to build. To manage multi-item settings, shared-parameter multi-agent policies are frameworks that enable policies to generalize across variable-length and permutationally inconsistent sets while accounting for shared resource constraints [1][5]. To encode the sets in a permutation-invariant way, we use the DeepSets encoder framework [2]. Demand forecasting has played a vital role in inventory control. Ensemble and bootstrap-based LSTM models provide probabilistic forecasts that capture temporal structure and uncertainty [3]. This project adopts such an approach by combining permutation-invariant state encodings, shared-parameter policies, and forecast-aware observations for perishable products used in restaurant inventory.

3 Data

Historical demand and contextual features are derived from the public Kaggle restaurant sales dataset [6], providing per-restaurant, per-dish daily demand with covariates (calendar, promotions, weather). Ingredient lead-times and shelf-lives are synthetically generated with an LLM [7] to define environment dynamics.

4 Problem Formulation

The restaurant inventory problem is modeled as a discrete-time Markov Decision Process (MDP) with daily decision intervals. At each day t, the agent observes the current inventory state, pending orders, demand forecasts, and available resources, then chooses order quantities $\mathbf{q}_t = (q_{1,t}, \ldots, q_{N,t})$ for N ingredients. Orders for ingredient i arrive after a lead time of ℓ_i days. The objective is to maximize expected discounted profit: (Appendix Eq. (1)). where π denotes the policy and $\gamma = 0.995$ reflects the long-horizon nature of inventory decisions.

State Representation. The state $\mathbf{s}_t$ combines global context and per-item features: (Appendix Eq. (2)) The global component uses cyclical sin/cos encoding for temporal features: (Appendix Eq. (3)) where B_t/B_0 is the budget ratio and used_t/C is the capacity utilization (on-hand inventory divided by total capacity). Capacity C is a fixed environment parameter (storage limit) used only in the transition constraints; the agent observes utilization but not the raw capacity value. Each item token (Appendix Eq. (4)) contains normalized cost ($c_i/100$), shelf-life ($L_i/10$), lead time ($\ell_i/5$), value contribution ($v_i/100$), an age-structured inventory vector $\mathbf{I}_{i,t}^{\text{age}} \in \mathbb{R}^{L^{\max}}$ (with $L^{\max} = 365$, normalized by 50), a pending-order pipeline $\mathbf{P}_{i,t} \in \mathbb{R}^{\ell^{\max}}$ indexed by days-to-arrival (with $\ell^{\max} = 30$, normalized by 50), and demand forecast statistics $(\hat{\mu}_{i,t}, \hat{\sigma}_{i,t})$ from the LSTM ensemble, normalized by the frozen initial expected demand. The per-item feature dimension is $D = 4+365+30+2 = 401$.

Additionally, a per-item *demand scale* $\hat{\mu}_{i,t}$ is provided, containing the LSTM forecast mean directly. This anchors the policy's action parameterization around the expected demand from the learned forecast model.

Action Space and LogNormal Policy. The policy outputs LogNormal-distributed actions (Appendix Eq. (5)–(6)): a log-space adjustment $\delta_{i,t}$ around forecast $\hat{\mu}_{i,t}$, with learned σ clamped to $[-3, 0.5]$. This ensures $q_{i,t} > 0$ by construction. Orders exceeding budget are rejected; orders exceeding capacity are scaled proportionally (Appendix Eq. (8)).

Reward Function. Reward (Appendix Eq. (9)) captures profit minus cost-weighted spoilage ($\kappa = 5$) and value-weighted shortage ($\lambda = 5$). Bankruptcy (budget ≤ 0) terminates the episode. Budget evolves via margin $m \in [0.3, 0.5]$ (Appendix Eq. (10)). Domain randomization: budget days $[20, 30]$, initial inventory days $[5, 10]$.

Warmup Phase. At reset, W days of demand are sampled to populate the LSTM history and compute frozen initial expected demand $\bar{d}_i^{\text{init}}$ for feature scaling.

Transition Dynamics. At each step: (1) pipeline orders arrive into age bucket 0 (overflow becomes spoilage); (2) inventory ages, oldest bucket expires; (3) dish demand is sampled from $d_k \sim \text{Poisson}(\lambda_k)$ where $\lambda_k = \text{base_rate}_k \times \prod m_j$ uses empirical multipliers for day-of-week, week-of-year, weather, and promotions; (4) demand fulfilled FIFO; (5) LSTM forecast updated.

5 Demand Forecasting with Bootstrapped LSTM Ensemble

The policy receives forecasts from an ensemble of M independent Gaussian LSTMs trained on bootstrapped samples. This architecture provides both point estimates and uncertainty quantification.

5.1 Model Architecture

Each ensemble member is a three-layer LSTM with Gaussian output: **Input:** concatenation of demand window $[d_{t-W}, \ldots, d_{t-1}]$ and context features $\mathbf{c}_t$ at each timestep; **context features (18 dims):** price, promotion flag, event flag, weather one-hot (4), temporal sin/cos (4), restaurant type one-hot (4), meal type one-hot (3); **LSTM:** hidden dimension 64, 3 layers, dropout 0.4; **output head:** MLP producing $(\mu, \log \sigma)$ for Gaussian likelihood.

The model is trained with Gaussian negative log-likelihood loss: (Appendix Eq. (11)) The model uses *running normalization* (Appendix Eq. (12)) to generalize across items. The ensemble provides uncertainty decomposition (Appendix Eq. (13)–(16)); per-item features include $\hat{\mu}_i$ and $\sigma_{\text{total},i}$, normalized by frozen initial expected demand.

5.2 Two-Stage Training

The LSTM is trained on synthetic trajectories from the simulator's Poisson demand model (not raw data), ensuring alignment with environment dynamics. Stage 1 computes per-item base rates and empirical multipliers from historical data; Stage 2 trains the LSTM ensemble on generated trajectories (Appendix Table (2)).

5.3 Training Results

Each ensemble member converges to $R^2 \approx 0.89$ on training data (164,400 samples). On held-out restaurants: mean $R^2 = 0.63$, with 78.6% of predictions within 2σ bounds (Figure 2). The validation gap reflects cross-restaurant generalization challenges, but calibrated uncertainty ensures reliable confidence estimates.

6 Baseline Policies

To benchmark the learned policy, we evaluate classical inventory heuristics adapted to the forecast-aware observation structure: Order-Up-To policies [8], (s, S) reorder-point policies [9, 10], and

Demand Matching. The policy setup is shown in Appendix Section (C). All baselines use the LSTM forecast features: the expected demand $\hat{\mu}_i$ is directly from the demand_scale observation field.

6.1 Results

We evaluate each policy over 5 episodes of 365 days across 50 restaurants. Table (Appendix Table (3)) summarizes the aggregated results.

Key Findings. Order-Up-To policies outperform (s, S) reorder-point policies due to continuous ordering (vs. conditional reordering which causes higher shortages). Higher safety factors z reduce shortage at the cost of increased spoilage. Demand Matching with a $1.2\times$ buffer achieves competitive reward with minimal shortage but higher capacity utilization. These baselines provide a performance floor for the learned PPO policy.

7 Proximal Policy Optimization Implementation

We employ Proximal Policy Optimization (PPO) [11] as the learning algorithm, implemented via Stable-Baselines3 [12] with a custom policy class tailored to the multi-item inventory structure. The architecture of the policy network is shown in Appendix Section (B).

7.1 LogNormal Action Distribution

As described in Section 4, the policy outputs LogNormal-distributed actions. The log-probability includes a Jacobian correction: (Appendix Eq. (22)), and entropy includes the mean term (Appendix Eq. (23)). The shared $\log \sigma$ is initialized to -1.5 and clamped to $[-3, 0.5]$. The environment pads observations and actions to $N_{\max} = 60$; invalid entries are masked in gradient updates and zeroed after sampling.

7.2 PPO Objective and Hyperparameters

The policy is trained using the clipped surrogate objective [11]: (Appendix Eq. (24)) where $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$ is the probability ratio, $\hat{A}_t$ is the GAE advantage estimate, and $\epsilon = 0.2$ is the clip range. The full loss for parameter-shared actor-critic systems (like ours since the item embeddings are pooled global state are shared between actor and critic) includes value function and entropy terms: (Appendix Eq. (25)) with $c_V = 0.5$ and $c_H = 0.01$.

Hyperparameters. Table (Appendix Table (4)) summarizes the training configuration.

The high γ and GAE λ reflect long-horizon inventory decisions; target KL (0.05) prevents destabilizing updates.

7.3 Training Procedure

Training uses `SubprocVecEnv` with 40 training restaurants (80/20 split); the shared policy generalizes via permutation-invariant architecture. Rewards are normalized (`VecNormalize`, `norm_reward=True`); observations use environment-applied scaling. Episodes are 365 days.

8 Results

Training curves for the PPO agent are shown in Appendix Figure 3. The agent was trained for 1000 episodes (approximately 14.6M timesteps) across 40 training restaurants.

8.1 Training Dynamics

Reward and profit (Figure 3a-b) show steady improvement over 500 episodes before plateauing. Shortage decreases while spoilage increases (Figure 3c-d), indicating the policy learns to over-order—a behavior warranting hyperparameter tuning. Capacity utilization (Figure 3e) shows oscillatory patterns from episode 500, likely due to synchronized episode terminations interacting with `VecNormalize`'s running statistics.

8.2 Validation Performance

Table 1 and Figure 1 compare PPO against baselines on 10 held-out validation restaurants (5 episodes, 365 days each).

Table 1: Validation results: PPO vs baselines on held-out restaurants.

Policy	**Reward**	**Profit**	**Spoil**	**Short**
Demand Match 1.2×	1,721	17,906	215	6
PPO (trained)	**1,606**	**22,065**	**50**	175
Demand Match 1.0×	1,110	16,839	45	185
Order-Up-To z=2.0	177	17,059	651	200
(s, S) z=2.0	−916	15,368	6	706

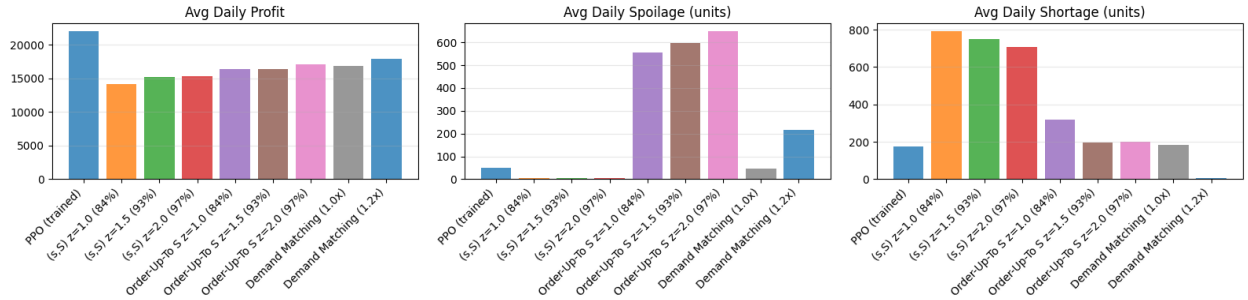


Figure 1: Validation: PPO achieves highest profit with lowest spoilage among continuous-ordering policies.

PPO achieves the highest profit ($22,065 vs next-best $17,906) with **lowest spoilage** among continuous-ordering policies (50 vs 215–651 units). Demand Matching 1.2× achieves highest reward via very low shortage (6 units), but PPO's profit margin offsets its moderate shortage (175 units).

8.3 Discussion

We expected PPO to learn demand patterns that reduce spoilage relative to safety-stock heuristics; we did not anticipate it would also achieve the highest profit despite accepting more stockouts. PPO generalizes effectively to held-out restaurants, reducing spoilage by 77% vs. Order-Up-To baselines

(50 vs. 215–651 units). The policy implicitly learns that spoilage is more costly than shortage: spoiled inventory incurs both purchasing cost and penalty, while stockouts only lose margin. This asymmetry, combined with the cost-weighted spoilage penalty ($\kappa = 5$), incentivizes slight under-ordering. No bankruptcies occurred across all policies.

9 Limitations and Future Work

Units and Interpretability. The current implementation uses unitless quantities for inventory, costs, and rewards. Incorporating explicit units (e.g., kg, ml) would improve interpretability and enable direct comparison with real-world operational metrics.

Hyperparameter Tuning. Training curves show increasing spoilage as the policy learns to avoid shortage. Achieving a better balance requires tuning: (1) reward weights κ/λ to penalize spoilage more heavily relative to shortage; (2) training hyperparameters such as learning rate, entropy coefficient c_H, and GAE λ; (3) architecture hyperparameters including hidden dimensions and pooling strategies. Systematic hyperparameter search was not performed in this work.

Production Planning. The current model treats ingredients as the decision variable. Extending to joint production planning—where the agent decides both dish preparation quantities and ingredient orders—would capture labor costs and multi-echelon coordination [13].

Sim-to-Real Gap. Demand is modeled via Poisson distributions with empirical multipliers for seasonality and context. Real restaurant demand exhibits more complex dynamics (e.g., substitution effects, social trends, competitor actions). Bridging the sim-to-real gap requires richer demand models and validation on real-world operational data.

10 Conclusion

We presented a deep reinforcement learning framework for restaurant inventory management of perishable goods. The PPO agent, using a DeepSets-based permutation-invariant architecture with LogNormal action distributions, learns to jointly optimize ordering decisions across multiple ingredients under budget and capacity constraints. Integrated LSTM demand forecasts provide uncertainty-aware observations that anchor the policy's decisions.

On held-out validation restaurants, the learned policy achieves the highest profit among all methods while maintaining the lowest spoilage among continuous-ordering policies. These results demonstrate that deep RL can effectively learn long-horizon trade-offs in high-dimensional perishable inventory domains where classical methods are intractable. Future work should address hyperparameter tuning for improved spoilage-shortage balance, extension to joint production planning, and validation on real-world operational data.

11 Code Availability

The data and code for this paper is available at `https://github.com/arnava13/RL-Restaurant-Inventory`.

References

[1] Nazneen N. Sultana, Hardik Meisheri, Vinita Baniwal, Somjit Nath, Balaraman Ravindran, and Harshad Khadilkar. Reinforcement learning for multi-product multi-node inventory management in supply chains. *arXiv preprint arXiv:2006.04037*, 2020.

[2] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabás Póczos, Ruslan Salakhutdinov, and Alexander J. Smola. Deep sets. *CoRR*, abs/1703.06114, 2017. URL `http://arxiv.org/abs/1703.06114`.

[3] Aman Singh, Ravi Kumar, Abhishek Gupta, and Joonho Lee. Deep reinforcement learning for inventory control: A roadmap. *Expert Systems with Applications*, 233:120934, 2023. doi: 10.1016/j.eswa.2023.120934.

[4] Rui Zhang, Min Xu, Xiang Lin, Jian Chen, and Keqiang Li. Deep inventory management. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 2023.

[5] Amandeep Kaur and Gyan Prakash. Adaptive inventory strategies using deep reinforcement learning for dynamic agri-food supply chains. *arXiv preprint arXiv:2507.16670*, 2025.

[6] Alexand Chen. Restaurant sales report 2024–2025, 2025. URL `https://www.kaggle.com/datasets/alexandchen/restaurant-sales-report-2024-2025`. Kaggle dataset, accessed December 2025.

[7] Anthropic. Claude opus 4.5, 2025. URL `https://www.anthropic.com/news/claude-opus-4-5`. Accessed December 2025.

[8] Metin Ülku. Base-stock policies and (s,s) systems. Technical report, University of Texas at Dallas, 2010. URL `https://personal.utdallas.edu/~metin/Or6302/Folios/ombasestock.pdf`.

[9] John Doe. Inventory models — lecture 12. Technical report, Massachusetts Institute of Technology, 2002. URL `https://dspace.mit.edu/...` 2-853 Manufacturing Systems I — Analytical Methods and Flow Models.

[10] Wikipedia contributors. Reorder point, 2025. URL `https://en.wikipedia.org/wiki/Reorder_point`. Accessed January 2025.

[11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017. URL `http://arxiv.org/abs/1707.06347`.

[12] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL `http://jmlr.org/papers/v22/20-1364.html`.

[13] Alvaro Maggiar, Sohrab Andaz, Akhil Bagaria, Carson Eisenach, Dean Foster, Omer Gottesman, and Dominique Perrault-Joncas. Structure-informed deep reinforcement learning for inventory management. *arXiv preprint arXiv:2507.22040*, 2025.

A Equations

$$\max_{\pi} \; \mathbb{E}_{\pi}\left[\sum_{t=0}^{T} \gamma^t R_t\right], \tag{1}$$

$$\mathbf{s}_t = \left(\mathbf{s}_t^{\text{global}}, \{\mathbf{z}_{i,t}\}_{i=1}^{N}, \{\hat{\mu}_{i,t}\}_{i=1}^{N}\right). \tag{2}$$

$$\mathbf{s}_t^{\text{global}} = \left(\frac{B_t}{B_0}, \frac{\text{used}_t}{C}, \sin\left(\frac{2\pi\text{dow}}{7}\right), \cos\left(\frac{2\pi\text{dow}}{7}\right), \sin\left(\frac{2\pi\text{woy}}{52}\right), \cos\left(\frac{2\pi\text{woy}}{52}\right)\right) \in \mathbb{R}^6 \tag{3}$$

$$\mathbf{z}_{i,t} = \left(\tilde{c}_i, \tilde{L}_i, \tilde{\ell}_i, \tilde{v}_i, \tilde{\mathbf{I}}_{i,t}^{\text{age}}, \tilde{\mathbf{P}}_{i,t}, \tilde{\mu}_{i,t}, \tilde{\sigma}_{i,t}\right) \tag{4}$$

$$\log q_{i,t} \sim \mathcal{N}\left(\log(\hat{\mu}_{i,t} + \varepsilon) + \delta_{i,t},\ \sigma^2\right), \tag{5}$$

$$q_{i,t} = \exp\left(\log(\hat{\mu}_{i,t} + \varepsilon) + \delta_{i,t} + \sigma\epsilon\right), \quad \epsilon \sim \mathcal{N}(0,1). \tag{6}$$

$$\sum_i c_i q_{i,t} \le B_t \qquad \text{(budget)}, \tag{7}$$

$$\sum_i \left(I_{i,t} + \text{pending}_{i,t} + q_{i,t}\right) \le C \qquad \text{(capacity)}. \tag{8}$$

$$R_t = \frac{\underbrace{\sum_i v_i S_{i,t}}_{\text{revenue}} - \underbrace{\sum_i c_i q_{i,t}}_{\text{purchasing}} - \kappa \underbrace{\sum_i c_i \cdot \text{Spoil}_{i,t}}_{\text{spoilage (cost-weighted)}} - \lambda \underbrace{\sum_i v_i \cdot \text{Short}_{i,t}}_{\text{lost sales (value-weighted)}}}{\max\left(\sum_i c_i \bar{d}_i,\ 1\right)}, \tag{9}$$

$$B_{t+1} = B_t - \sum_i c_i q_{i,t} + m \cdot \text{revenue}_t, \tag{10}$$

$$\mathcal{L} = -\log \mathcal{N}(d_t \mid \mu, \sigma^2). \tag{11}$$

$$\tilde{d}_\tau = \frac{d_\tau - \bar{d}_{\text{window}}}{\max(\sigma_{\text{window}}, 1)}, \quad \tau \in [t - W, t]. \tag{12}$$

$$\text{Forecast mean: } \hat{\mu} = \frac{1}{M}\sum_{m=1}^{M} \mu^{(m)}, \tag{13}$$

$$\text{Aleatoric (inherent noise): } \sigma_{\text{ale}} = \frac{1}{M}\sum_{m=1}^{M} \sigma^{(m)}, \tag{14}$$

$$\text{Epistemic (model uncertainty): } \sigma_{\text{epi}} = \text{std}(\{\mu^{(m)}\}_{m=1}^{M}), \tag{15}$$

$$\text{Total uncertainty: } \sigma_{\text{total}} = \sqrt{\sigma_{\text{ale}}^2 + \sigma_{\text{epi}}^2}. \tag{16}$$

$$S_i = \hat{d}_i(L_i + R) + z \cdot \sigma_i \cdot \sqrt{L_i + R}, \qquad q_i = \max(0, S_i - \mathrm{IP}_i), \tag{17}$$

$$s_i = \hat{d}_i \cdot L_i + z \cdot \sigma_i \cdot \sqrt{L_i}, \tag{18}$$

$$S_i = \hat{d}_i(L_i + R) + z \cdot \sigma_i \cdot \sqrt{L_i + R}, \tag{19}$$

$$q_i = \begin{cases} \max(0, S_i - \mathrm{IP}_i) & \text{if } \mathrm{IP}_i \le s_i \\ 0 & \text{otherwise} \end{cases} \tag{20}$$

$$z_i \sim \mathcal{N}(\mu_i(s), \sigma^2), \quad q_i = \exp(z_i), \tag{21}$$

$$\log \pi(q_i \mid s) = \log \mathcal{N}(z_i \mid \mu_i(s), \sigma^2) - \log q_i, \tag{22}$$

$$H(\pi(\cdot \mid s)) = \sum_i m_i \left[\tfrac{1}{2}\log(2\pi e \sigma^2) + \mu_i(s)\right], \tag{23}$$

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t \left[\min\Big(r_t(\theta)\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\right], \tag{24}$$

$$L(\theta) = -L^{\mathrm{CLIP}}(\theta) + c_V\, L^{\mathrm{VF}}(\theta) - c_H\, H(\pi_\theta), \tag{25}$$

$$\mathbf{e}_i = \phi(\mathbf{z}_{i,t}) = \mathrm{ReLU}\big(W_2\, \mathrm{ReLU}(W_1 \mathbf{z}_{i,t} + b_1) + b_2\big), \tag{26}$$

$$\mathbf{p}_{\mathrm{sum}} = \sum_i m_i\, \mathbf{e}_i, \tag{27}$$

$$\mathbf{p}_{\mathrm{mean}} = \mathbf{p}_{\mathrm{sum}} \big/ \max\big(1, \textstyle\sum_i m_i\big), \tag{28}$$

$$\mathbf{p}_{\mathrm{max}} = \max_i \big\{\mathbf{e}_i : m_i = 1\big\}, \tag{29}$$

B Policy Architecture

The DeepSets actor-critic policy processes variable-length item sets through three stages:

Item Encoder. Each item token $\mathbf{z}_{i,t} \in \mathbb{R}^{401}$ is mapped to an embedding $\mathbf{e}_i \in \mathbb{R}^{64}$ via a 2-layer MLP (Eq. (26)).

Masked Pooling. Item embeddings are aggregated into a permutation-invariant global state using masked sum, mean, and max pooling (Eq. (27)–(29)), concatenated with global features $\mathbf{s}_t^{\mathrm{global}}$.

Actor/Critic Heads. The actor head takes each item embedding $\mathbf{e}_i$ concatenated with the pooled global state $\mathbf{s}_g$ and outputs $\mu_i(s)$ for the LogNormal distribution. The critic head takes only $\mathbf{s}_g$ and outputs a scalar value estimate $V(s)$.

Hidden dimensions: item encoder 64, actor 128, critic 128.

C Baseline Policy Setup

All baseline policies operate on the same observation structure as the learned policy. At each step, baselines extract:

- **Inventory position** IP_i: on-hand inventory plus pending orders for item i
- **Forecast mean** $\hat{d}_i = \hat{\mu}_i$: from the `demand_scale` observation field
- **Forecast std** $\sigma_i = \hat{\sigma}_i$: from the LSTM ensemble uncertainty

Policies differ in how they compute order quantities from these inputs (Eq. (17)–(20)).

D Tables

Table 2: LSTM ensemble training configuration.

Parameter	**Value**
Restaurants	50
Trajectories per restaurant	5
Trajectory length	180 days
Window size W	60 days
Ensemble size M	5 models
Hidden dimension	64
LSTM layers	3
Dropout	0.4
Learning rate	10^{-3}
Weight decay	10^{-5}
Epochs	50
Batch size	4096

Table 3: Full validation results (5 episodes $\times$ 10 validation restaurants, 365 days each).

Policy	**Avg Reward**	**Std**	**Profit**	**Spoil**	**Short**
Demand Match 1.2$\times$	1,721	599	17,906	215	6
PPO (trained)	1,606	550	22,065	50	175
Demand Match 1.0$\times$	1,110	473	16,839	45	185
Order-Up-To z=2.0	177	2,151	17,059	651	200
Order-Up-To z=1.5	66	2,398	16,408	599	195
Order-Up-To z=1.0	−182	2,742	16,380	555	320
(s, S) z=2.0	−916	179	15,368	6	706
(s, S) z=1.5	−1,124	180	15,173	6	752
(s, S) z=1.0	−1,367	138	14,194	5	793

Table 4: PPO hyperparameters used in training.

Parameter	**Value**
Learning rate	1×10^{-4}
Discount factor γ	0.995
GAE λ	0.95
Rollout steps per env	2048
Minibatch size	4096
PPO epochs per update	5
Clip range ϵ	0.2
Target KL	0.05
Value coefficient c_V	0.5
Entropy coefficient c_H	0.01
Max gradient norm	0.5

E Graphs

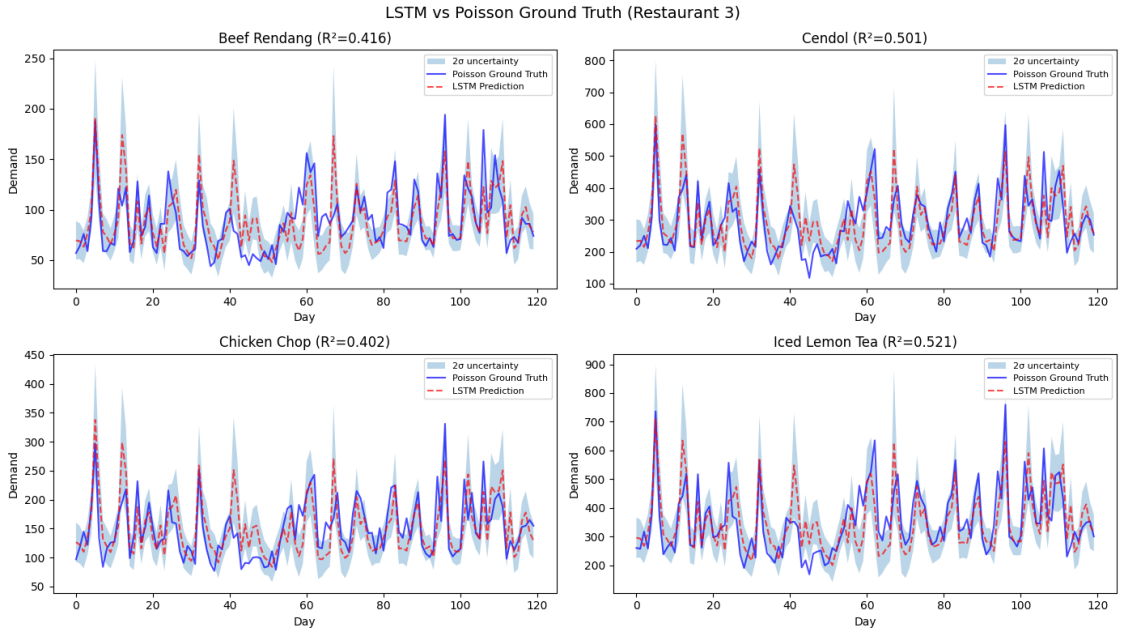
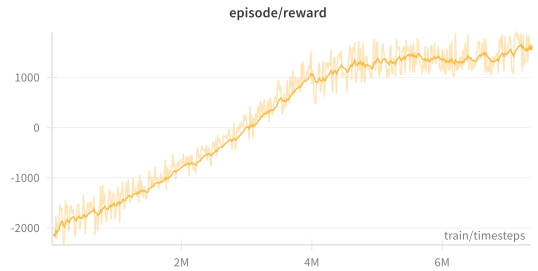
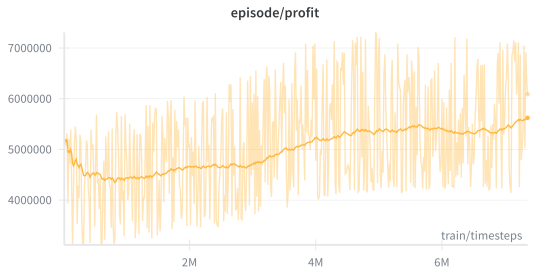


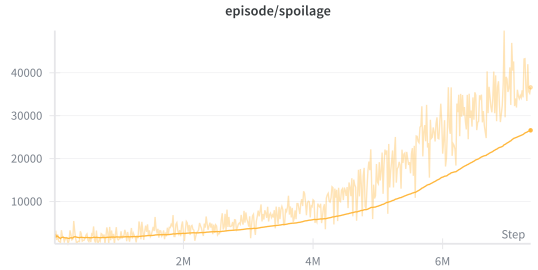
Figure 2: LSTM ensemble predictions vs. Poisson ground truth on a held-out restaurant (Cafe, 9 items). Blue: actual demand trajectory; red dashed: LSTM prediction; shaded region: 2σ uncertainty. The model captures weekly seasonality patterns while maintaining calibrated uncertainty bounds.



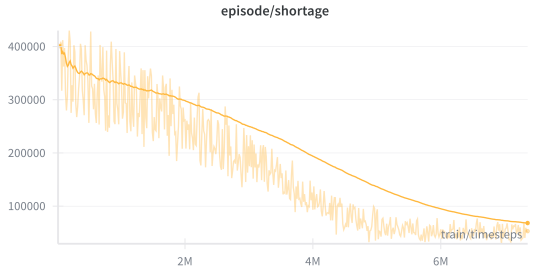
(a) Episode reward



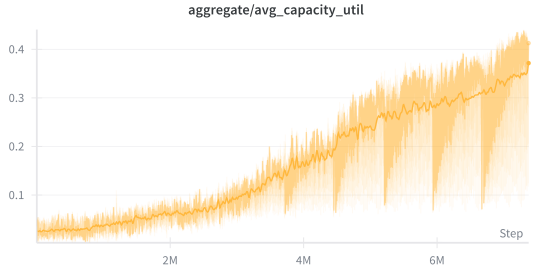
(b) Episode profit



(c) Episode spoilage



(d) Episode shortage



(e) Capacity utilization

Figure 3: PPO training curves over 1000 episodes (∼14.6M timesteps) across 40 training restaurants. (a) Normalized episode reward shows learning progress. (b) Raw profit trajectory. (c-d) Spoilage and shortage trade-off dynamics. (e) Capacity utilization convergence.